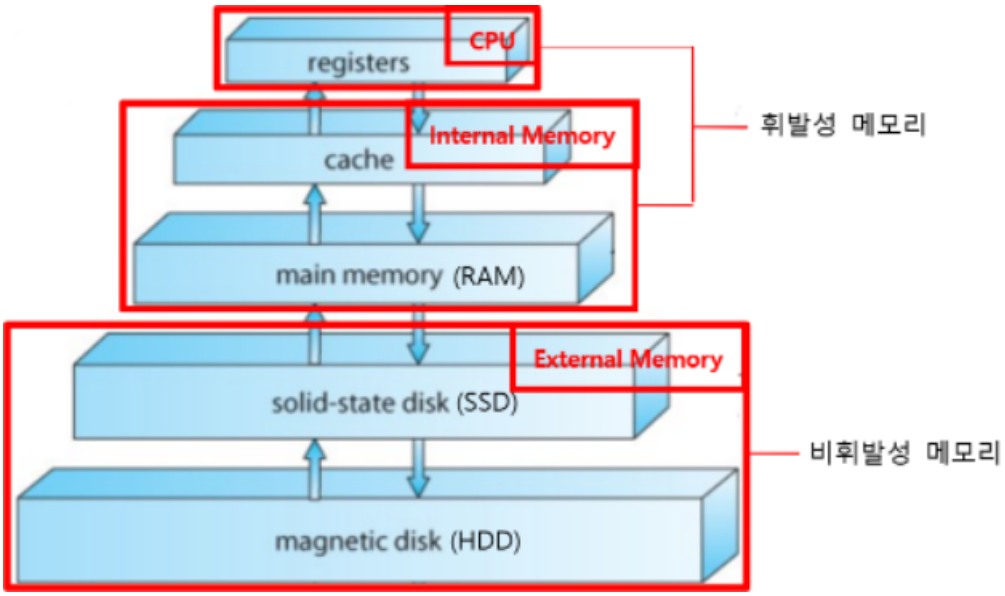
이전까지는 데이터 베이스를 사용하는 방법에 대해서 알아 보았다.

즉, Postgre SQL과 같은 DB를 사용하는 방법과, 어떻게 데이터를 저장하는 것이 좋은 방법인지와 같은 Logical 한 방법론들을 배웠었다.

여기서 부터는 이 데이터 베이스를 실제로 Phisycal하게 어떻게 설계할 것인지 알아보도록 하자.

#Storage Hierarchy

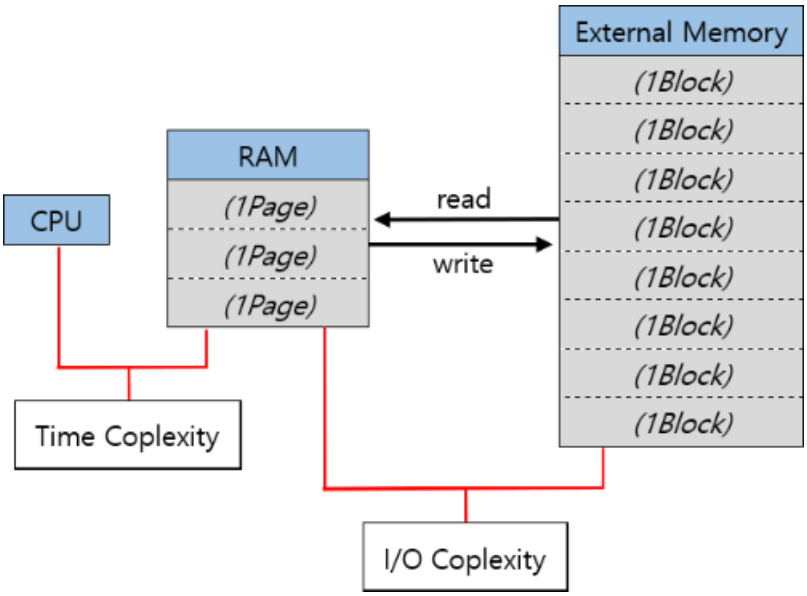
Data Base에 저장되는 Data들은 사용될 때에는 Main Memory에 올라오지만 평소에는 External Memory에 저장되어 보관된다.



위의 그림을 보면 External Memory는 구조적으로 CPU에서 가장 멀리 떨어져 있는 것을 확인할 수 있는데, 이 때문에 CPU에서 External Memory로 Access하는데에 시간이 매우 많이 사용된다.

즉, 우리가 Data Base를 Physical하게 설계할 때, 이 Access의 횟수를 관리하는 것이 가장 중요한 일 중 하나가 된다.

#I/O Complexity



이 External Memory에 접근하는 횟수를 측정하는 방법론 중 하나가 I/O Complexity이다.

알고리즘 시간에서는 우리가 짠 코드의 시간 복잡도를 측정하기 위해 Time Coplexity를 사용했다.

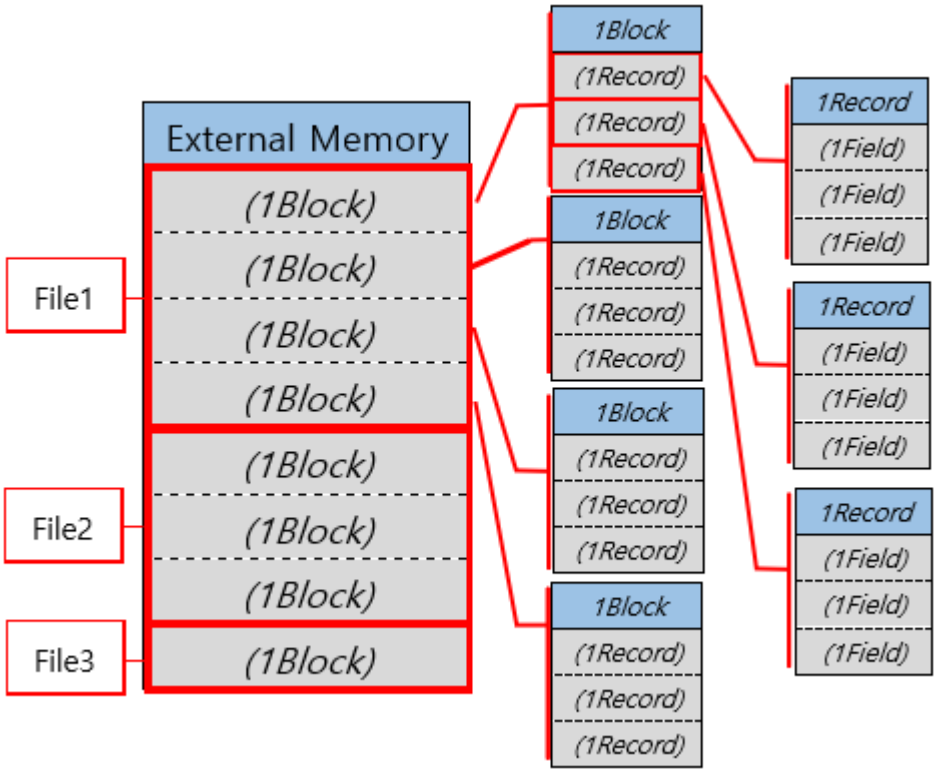
Time Complexity는 CPU와 RAM사이의 데이터를 주고 받는 횟수와 CPU의 Basic Operation 횟수 등을 측정하는데 사용했던 것이었다면,
I/O Complexity는 RAM과 External Memory사이에서 데이터 Block을 주고받는 횟수를 측정하는데 사용하는 것이다.

> **Block, Page**
> CPU와 RAM은 Byte 단위로 데이터를 주고받는 것이 가능했다.
> 하지만 RAM과 External Memory는 한꺼번에 많은 양의 데이터를 처리하기 위해서 정해진 Block(Page)단위로 데이터를 주고받는다.

File Organization

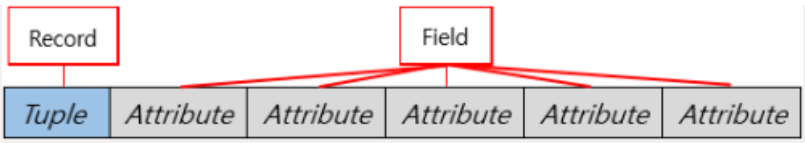
우선 데이터 베이스는 External Memory에 저장되어 사용된다. 즉, 결국엔 실제 Data들은 Physical하게 Memory에 File의 형태로 저장되어 OS가 관리하게 된다.

1. File Structure



File은 OS가 External Memory의 Data를 관리하기 위해 만든 구조이다. File은 위와 같은 구조로 저장된다.

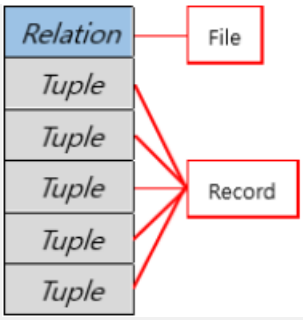
1) Record: Sequence Of Field



Field를 우리가 저장할 Data라고 한다면 Record는 Field로 구성된 Data Set이다.

즉, 우리가 전에 보았던 Relational DataBase모델에서 Tuple과 Record는 비슷한 역할을 하고 있다고 생각할 수 있다.

2) File: Sequence Of Record



File은 위의 Record를 기본단위로 하는 Record들의 모음이라고 볼 수 있다.

즉, 우리가 이전에 보았던 Relational DataBase모델에서 Table과 File은 비슷한 역할을 하고 있다고 생각할 수 있다.

3) Block

> 우리는 file을 사용할 때 결국 RAM에 올려서 사용해야 한다. 즉, External Memory에 존재하는 Data를 Read해야 한다는 것이다.
> 이때 일정한 크기의 Record의 단위대로 Read하게 되는데 이 단위를 Block이라고 한다.
>
> 즉, Block은 Sequence Of Record라고도 할 수 있을 것이다.
> 또한, File은 Sequence Of Block이라고도 할 수 있을 것이다.

@참고

> **자주 사용하는 I/O Complexity 약어**
>
> 1. N: Input File의 Record 개수
> 2. M: Main Memory에 얼마나 탑재될 수 있는 Record의 개수
> (Main Memory의 Capacity)
> 3. B: 하나의 Block에 탑재 될 수 있는 Record의 개수
> (Block Size)
>
> ---
>
> **주요 I/O Complexity**
>
> $$Linear\ Cost = O(\frac{N}{B})$$
>
> $$Logarithmic\ Cost = O(\ log_B N\ )$$
>
> $$Nlog(N)\ Cost = O(\frac{N}{B} log_{\frac{M}{B}} \frac{N}{B})$$

---

2. File Organization

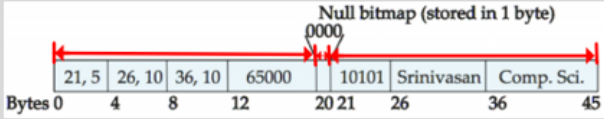
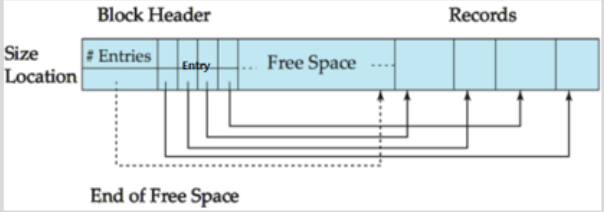
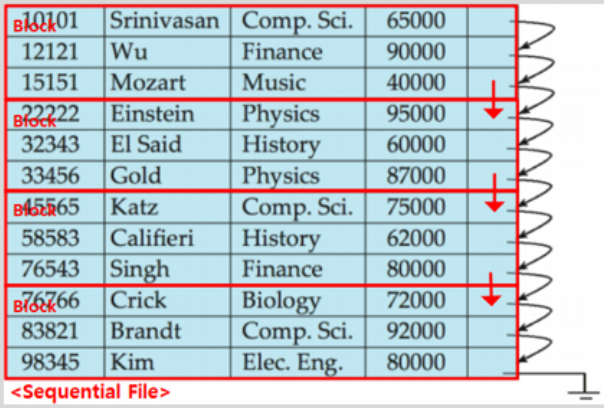
이제 Record들이 어떻게 Organize되어야 File이 효율적으로 관리될 수 있는지 I/O Complexity 관점에서 생각해 보자

1) Fixed Length Records

<table>
<tr><th></th><th>방법1</th><th>방법2</th><th>방법3</th></tr>
<tr><td>그림</td><td>
<table>
<tr><td>record 0</td><td>10101</td><td>Srinivasan</td><td>Comp. Sci.</td><td>65000</td></tr>
<tr><td>record 1</td><td>12121</td><td>Wu</td><td>Finance</td><td>90000</td></tr>
<tr><td>record 2</td><td>15151</td><td>Mozart</td><td>Music</td><td>40000</td></tr>
<tr><td>record 4</td><td>32343</td><td>El Said</td><td>History</td><td>60000</td></tr>
<tr><td>record 5</td><td>33456</td><td>Gold</td><td>Physics</td><td>87000</td></tr>
<tr><td>record 6</td><td>45565</td><td>Katz</td><td>Comp. Sci.</td><td>75000</td></tr>
<tr><td>record 7</td><td>58583</td><td>Califieri</td><td>History</td><td>62000</td></tr>
<tr><td>record 8</td><td>76543</td><td>Singh</td><td>Finance</td><td>80000</td></tr>
<tr><td>record 9</td><td>76766</td><td>Crick</td><td>Biology</td><td>72000</td></tr>
<tr><td>record 10</td><td>83821</td><td>Brandt</td><td>Comp. Sci.</td><td>92000</td></tr>
<tr><td>record 11</td><td>98345</td><td>Kim</td><td>Elec. Eng.</td><td>80000</td></tr>
</table>
</td><td>
<table>
<tr><td>record 0</td><td>10101</td><td>Srinivasan</td><td>Comp. Sci.</td><td>65000</td></tr>
<tr><td>record 1</td><td>12121</td><td>Wu</td><td>Finance</td><td>90000</td></tr>
<tr><td>record 2</td><td>15151</td><td>Mozart</td><td>Music</td><td>40000</td></tr>
<tr><td>record 11</td><td>98345</td><td>Kim</td><td>Elec. Eng.</td><td>80000</td></tr>
<tr><td>record 4</td><td>32343</td><td>El Said</td><td>History</td><td>60000</td></tr>
<tr><td>record 5</td><td>33456</td><td>Gold</td><td>Physics</td><td>87000</td></tr>
<tr><td>record 6</td><td>45565</td><td>Katz</td><td>Comp. Sci.</td><td>75000</td></tr>
<tr><td>record 7</td><td>58583</td><td>Califieri</td><td>History</td><td>62000</td></tr>
<tr><td>record 8</td><td>76543</td><td>Singh</td><td>Finance</td><td>80000</td></tr>
<tr><td>record 9</td><td>76766</td><td>Crick</td><td>Biology</td><td>72000</td></tr>
<tr><td>record 10</td><td>83821</td><td>Brandt</td><td>Comp. Sci.</td><td>92000</td></tr>
</table>
</td><td>
<table>
<tr><td>header</td><td></td><td></td><td></td><td>record1</td></tr>
<tr><td>record 0</td><td>10101</td><td>Srinivasan</td><td>Comp. Sci.</td><td>65000</td></tr>
<tr><td>record 1</td><td></td><td></td><td></td><td>record4</td></tr>
<tr><td>record 2</td><td>15151</td><td>Mozart</td><td>Music</td><td>40000</td></tr>
<tr><td>record 3</td><td>22222</td><td>Einstein</td><td>Physics</td><td>95000</td></tr>
<tr><td>record 4</td><td></td><td></td><td></td><td>record6</td></tr>
<tr><td>record 5</td><td>33456</td><td>Gold</td><td>Physics</td><td>87000</td></tr>
<tr><td>record 6</td><td></td><td></td><td></td><td>NULL</td></tr>
<tr><td>record 7</td><td>58583</td><td>Califieri</td><td>History</td><td>62000</td></tr>
<tr><td>record 8</td><td>76543</td><td>Singh</td><td>Finance</td><td>80000</td></tr>
<tr><td>record 9</td><td>76766</td><td>Crick</td><td>Biology</td><td>72000</td></tr>
<tr><td>record 10</td><td>83821</td><td>Brandt</td><td>Comp. Sci.</td><td>92000</td></tr>
<tr><td>record 11</td><td>98345</td><td>Kim</td><td>Elec. Eng.</td><td>80000</td></tr>
</table>
</td></tr>
<tr><td>설명</td><td>File의 각 Record들이 항상 비어있지 않고 Sequential하도록 유지시키는 방법</td><td>File의 각 Record들이 항상 비어있지 않도록만 하는 방법</td><td>Header를 추가하여 비어있는 Record의 위치를 마치 Singly Linked List와 같이 연결한 Free List를 사용하는 방법</td></tr>
<tr><td>Access</td><td>$O(1)$
Record의 Size가 일정하므로 우리가 접근할 Record의 위치는 바로 결정된다.</td><td>$O\left(\frac{N}{B}\right)$
Record의 번호를 알고 있더라도 위치가 결정되지 않으므로 모든 Block을 불러와 찾는다.</td><td>$O(1)$
Record의 번호만 알면 방법 1과 마찬가지로 바로 Access가 가능하다.</td></tr>
<tr><td>Insertion</td><td>$O(1)$
마지막 Record의 뒤에 새로운 Record를 넣으면 된다.</td><td>$O(1)$
마지막 Record의 뒤에 새로운 Record를 넣으면 된다.</td><td>$O(1)$
Header에 기록된 Record의 번호에 Insert를 수행하고 Header엔 Insert를 시행한 Record가 원래 가리켰던 Record의 번호를 넣는다.</td></tr>
<tr><td>Deletion</td><td>$O\left(\frac{N}{B}\right)$
삭제한 Record의 빈 자리를 뒤의 Record를 당겨와 메꿔준다.</td><td>$O(1)$
맨 마지막 Record를 불러와 삭제된 Record의 위치에 넣어준다.</td><td>$O(1)$
Header에 삭제할 Record의 번호를 넣고, 삭제할 Record의 번호엔 원래 Header에 기록되어 있던 Record의 번호를 넣는다.</td></tr>
</table>

> **(주의점)**
>
> 1. I/O Complexity는 불러와야 하는 Block의 개수에 따라 달라진다는 것을 기억하자.
> *(ex. 방법3의 Deletion은 Header가 위치한 Block과 삭제할 Record가 위치한 Block만 불러오면 된다.)*
>
> ---
>
> 2. 1~2번 방법의 Free Space는 마지막 Record의 뒤가 되고,
> 3번 방법의 Free Space는 Free List에 속한 Record가 된다.
>
> ---
>
> 3. 방법 3에서 만약 Header가 아닌 Free List의 마지막에 삭제할 Block의 Record 번호를 넣게되면 최악의 경우 `O(N)`의 I/O Complexity를 갖게 된다.

2) Variable Length Record

	그림	설명
Store Field in Record		Record를 3부분으로 나누어 관리한다. **1. Fixed Length Part** : (*Variable Length Part의 시작 Field번호, 길이*)를 기록해 둔다. **2. Null Bitmap** : (*Null 값을 가지는 Fixed Lengt Part의 번호*)를 기록해 둔다. **3. Variable Length Part** : (*Fixed Length Part에 해당하는 값*)을 기록해 둔다.
Store Record in Block		Block을 3부분으로 나누어 관리한다. **1. Block Header** : 맨 앞에는 Entry의 수가 기록되어 있다. : 그 뒤에는 Free Space의 마지막이 기록되어 있다. : 그 뒤부터는 각 Record의 시작과 끝이 기록되어 있다. **2. Free Space** : Block에서 비어있는 부분으로 Record를 기록할 수 있는 공간이다. **3. Records** : 실제 Record가 기록되는 부분이다.
Store Block in File		**1. Heap File** Header를 활용해 Block을 Random하게 배치해도 상관없어, 삽입/삭제가 많아도 영향 X **2. Sequential File** Search Key를 활용해 Block내에서 이 Search Key의 순서대로 Record의 Entry가 연결되도록 정렬해 놓은 File (마지막 Entry는 다른 Block의 Entry를 가리키도록 연결) **3. Hash File** Hash Function을 통해 저장할 Block의 위치를 결정해 놓는 File **Pointer** 원하는 조건에 따라 검색하기 편하도록 Index를 저장해둔 또 다른 형태의 File.

위의 Fixed Length Record일 경우와는 다르게 Variable Length Record일 경우 Record의 위치를 바로 결정할 수 없다.

즉, Fixed Length Record의 방법과 같이 Organization시킬 경우 Access할 때, 원하는 Record를 찾을 때 까지 모든 Block을 불러와야 하므로 최악의 경우 $O(\frac{N}{B})$의 I/O Complexity를 갖게된다.

이를 해결하기 위해 위의 그림과 같이 Record를 Organization시켜 보자.

	Store Field in a Record	Store Record in a Block	Store Block in a Sequential File
Access	$O(1)$ Fixed Length Part에서 원하는 Field의 위치를 받아 바로 Variable Length Part로 움직이면 된다.	$O(1)$ Block Header의 Entry에서 Record의 시작과 끝의 정보를 받아 바로 접근할 수 있다.	$O\left(log_2 \frac{N}{B}\right)$ Search Key의 순서대로 정렬되어 있으므로 Block에 대해 Binary Search를 수행할 수 있다.
Insertion	- (NULL값의 경우 NULL Bitmap을 활용)	$O(1)$ **Entry의 개수 변경,** **Free Space 변경** : 왼쪽에는 Entry를 추가하고, : 오른쪽에는 해당 Record를 추가한다.	$O(1)$ Search Key의 순서에 맞게 Block에 넣고, Entry 간의 연결을 조정해준다. 이때, **Free Space가 있는 경우** **:** 바로 Insertion을 수행한다. **Free Space가 없는 경우** **:** 넘친 Record에 대해서 Overflow Block을 새로 만들어 저장한다.
Deletion		$O(1)$ **Entry의 개수 변경,** **Entry 삭제,** : Free List를 활용해 빈 자리를 유지해도 된다. **Record 삭제** : 왼쪽의 Record를 당겨와 빈공간을 지워준다.	$O(1)$ 삭제된 Record가 포함된 Entry간의 Pointer Chain을 끊고 Entry간의 연결을 조정해 준다.

(주의점)

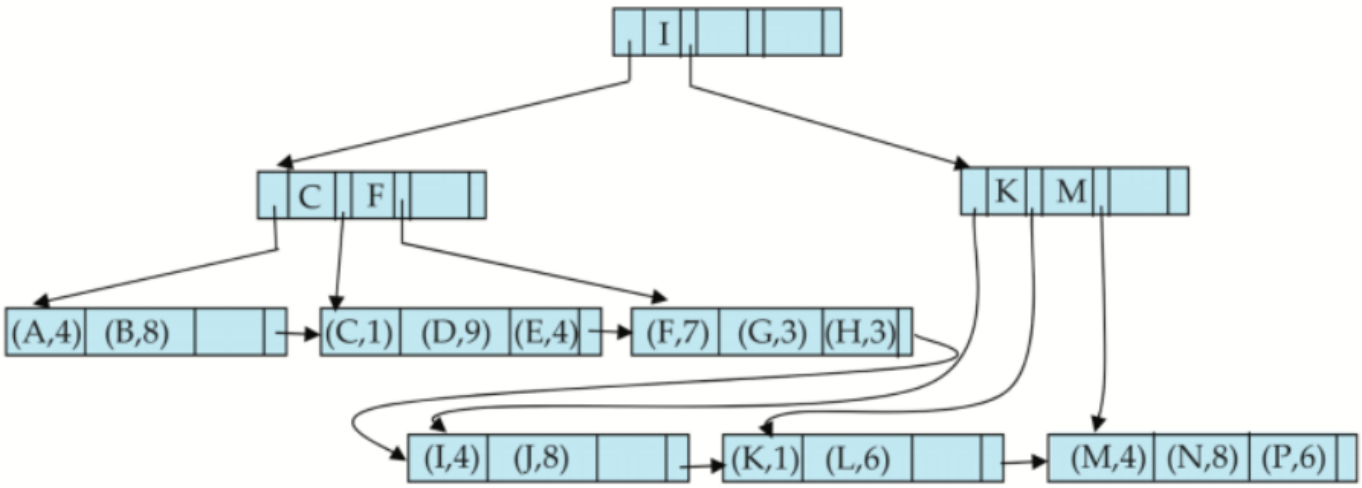
1.(Block) Block에서 Entry는 왼쪽에서 오른쪽으로, Record는 오른쪽에서 왼쪽으로 채워진다.
(Free Space는 항상 연속되도록 유지한다.)

2.(Block) Block에서 Entry를 삭제할 때, 만약 삭제된 Record부분을 비워둔다면 Internal Fragment가 발생할 수 있다.
(공간이 존재하지만 사용이 불가능한 상황)

3.(Block) Block에서 Entry를 삭제할 때, Record를 하나씩 당겨와 채워넣더라도 새로운 Block을 가져올 필요가 없으므로 I/O Complexity에 영향을 주지 않는다.

4.(File) 시간이 지나면 Entry간의 연결이 꼬여 Block간의 연결이 많아질 수 있기 때문에 한번씩 Sequential Order를 Reorganize할 필요가 있다.

3) B+Tree File Organization



위에서 보았듯이 File을 Organization 하는 방법은 Heap, Sequential, Hash등 다양한 방법이 있었다.

> 이 때, B+ Tree를 활용할 경우 Record들을 좀 더 효과적으로 보관할 수 있게된다.
>
> B+ Tree의 작동 원리는 다음장을 참고하면 된다.
> *(해당 B+ Tree와 유일하게 다른 점은 Leaf Node에 pointer가 아닌 Record 자체가 들어간다는 점이다.)*

File Access

우리는 위에서 Organized한 File을 사용하기 위해서는 Main Memory에 올려야 한다.

이 때, 우리가 어떤 프로그램을 사용하면서 Main Memory에 올릴 수 있는 데이터의 크기는 제한되어 있기 때문에 이를 효과적으로 사용하는 것이 중요하다.

즉, I/O Complexity를 최소화 하는 방법으로 Data Block을 Memory에 올렸다 내렸다 하는 방법을 정해야 한다는 것이다.

1. 용어

1) Buffer

> Main Memory의 일부로, Disk에 존재하는 Data Block들을 복사하여 Memory에 올려 놓는 부분을 의미한다.

2. Buffer Manager

> Buffer를 관리하는 프로그램을 의미한다.

---

2. Buffer 관리 전략

Disk의 메모리를 Buffer에 올리려고 할 때, Memory는 다음과 같은 동작을 통해 Buffer를 관리하게 된다.

1. *Buffer Manager에 현재 Buffer에 우리가 원하는 Data Block이 올라와 있는지 물어본다.*

2. *(올라와 있는 경우)*
   Buffer Manager는 해당 Data Block의 Address를 바로 알려준다.

3. *(올라와 있지 않은 경우)*
   - *Buffer에 빈 공간이 있을 때, 해당 Block을 바로 Buffer에 올려준다.*
   - *Buffer에 빈 공간이 없을 때, Victim을 선정해 Buffer를 비우고 해당 Block을 Buffer에 올려준다.*

4. *작업 완료 후 Memory에서 데이터를 변경하였을 경우 다시 Disk에 Written Back 해준다.*

(Victim이란? 원래 Buffer에 있던 Data Block중 지울 Data Block을 의미한다.)

이때, 이 Victim을 정하는 방법에 따라 I/O Complexity가 달라지게 된다. 따라서 더 나은 Victim을 정하기 위해, 상황(Query의 종류)에 따라 Victim의 선택 방법을 달리 사용하게 된다.

1) LRU (Least Recently Used)

> **1. 방법**
>
> 최근에 사용되지 않았던 Block을 먼저 Victim으로 정하는 방법이다.
>
> **2. 상황**
>
> Temporal(Spatial) Locality아이디어에 의해 주로 OS에서 이 방법을 사용한다.

2) MRU (Most Recently Used)

> **1. 방법**
>
> 가장 최근에 사용되었던 Block을 먼저 Victim으로 정하는 방법이다.
>
> **2. 상황**
>
> 주로 Join연산을 할 때 사용하게 된다.
>
> ---
>
> **(예시)**
>
> Memory의 buffer용량이 Block 3개일 때, Relation `r`과 `s`를 Join한다고 한다고 생각해보자.
>
> 먼저 `r`의 Block 1개와 `s`의 Block 2개를 올린다.
>
> 이때, `s`에 join할 Block들이 더 남아 있다면 `r`의 Block은 유지시키는 것이 좋을 것이다.
>
> 즉, `r`은 가장 먼저 Buffer에 들어왔지만 Buffer에 유지시키고 가장 나중에 들어온 `s`의 Block을 바꾸는 것이 더 현명한 선택일 것이다.

3)기타

> **1. 방법**
>
> 위의 방법 외에도 기타 여러 정보들을 복합적으로 생각해 Victim을 정할 수 있다.
>
> **2. 예시**
>
> file Header는 자주 사용되지 않더라도 Main Memory에 유지시키는 것이 좋다.